

IDEATION PHASE

Brainstorm & Idea Prioritization

Date: 19 June 2026

Team ID: -----

Project Name: AI-Enhanced Intrusion Detection System

Maximum Marks: 4 Marks

Brainstorm & Idea Prioritization Template

Brainstorming provides a collaborative environment that encourages innovative thinking and problem-solving. Through structured idea generation and prioritization, the team identifies the most feasible and impactful solution for addressing cybersecurity challenges.

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Team Members

- Sourabh Bhimrao Patil
- Project Guide / Mentor
- Cybersecurity Domain Experts
- Machine Learning Contributors

Problem Statement

Traditional Intrusion Detection Systems (IDS) rely heavily on predefined signatures and rules, making them ineffective against emerging cyber threats and zero-day attacks. There is a need for an intelligent system capable of detecting, classifying, and responding to network intrusions in real time using Artificial Intelligence.

Diagram 1: Problem Statement Identification

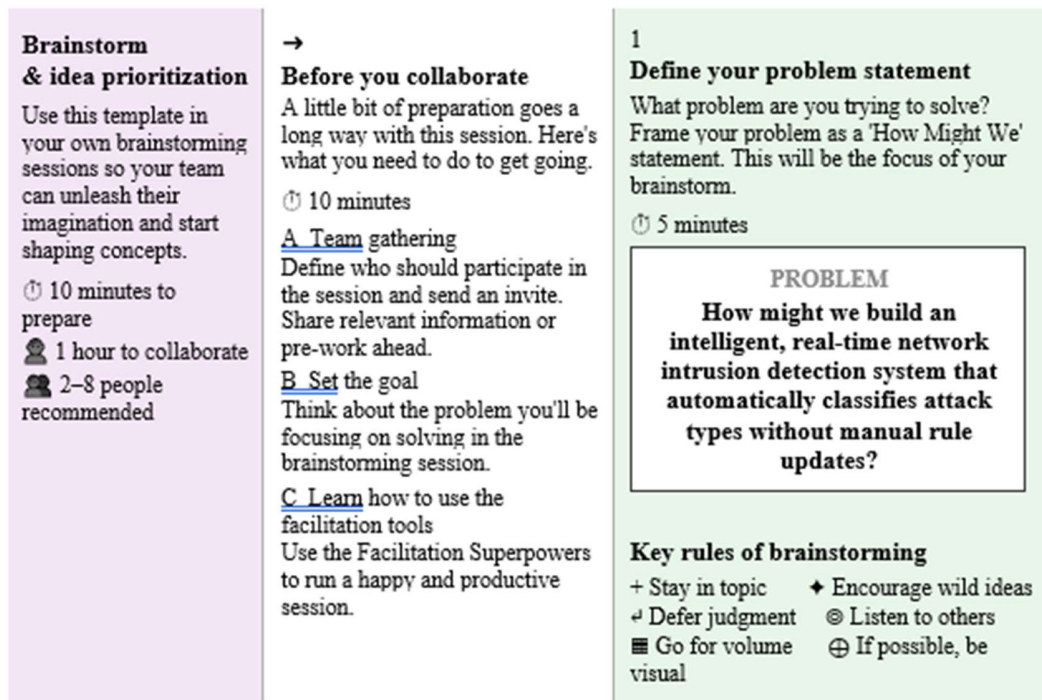


Figure 1: Problem Statement Selection

Step-2: Brainstorm, Idea Listing and Grouping

Brainstormed Ideas

Machine Learning Ideas

- Random Forest Intrusion Detection
- Decision Tree-Based Detection
- Support Vector Machine (SVM)
- Deep Learning-Based Detection

Security Monitoring Ideas

- Real-Time Threat Detection
- Automated Alert Management
- Attack Severity Classification

Data Processing Ideas

- NSL-KDD Dataset Analysis
- Feature Engineering
- Automated Data Preprocessing

Deployment Ideas

- REST API Integration
 - Cloud-Based Deployment
 - Security Analytics Dashboard
-

Idea Grouping

Group A – AI & Machine Learning

- Random Forest Classifier
- Support Vector Machine
- Deep Learning Models

Group B – Cybersecurity Monitoring

- Intrusion Detection
- Threat Classification
- Alert Management

Group C – User Interface

- Dashboard Monitoring
- Security Analytics
- Report Generation

Group D – Deployment & Integration

- REST API
- Cloud Infrastructure
- Real-Time Monitoring

Diagram 2: Brainstorming and Idea Grouping

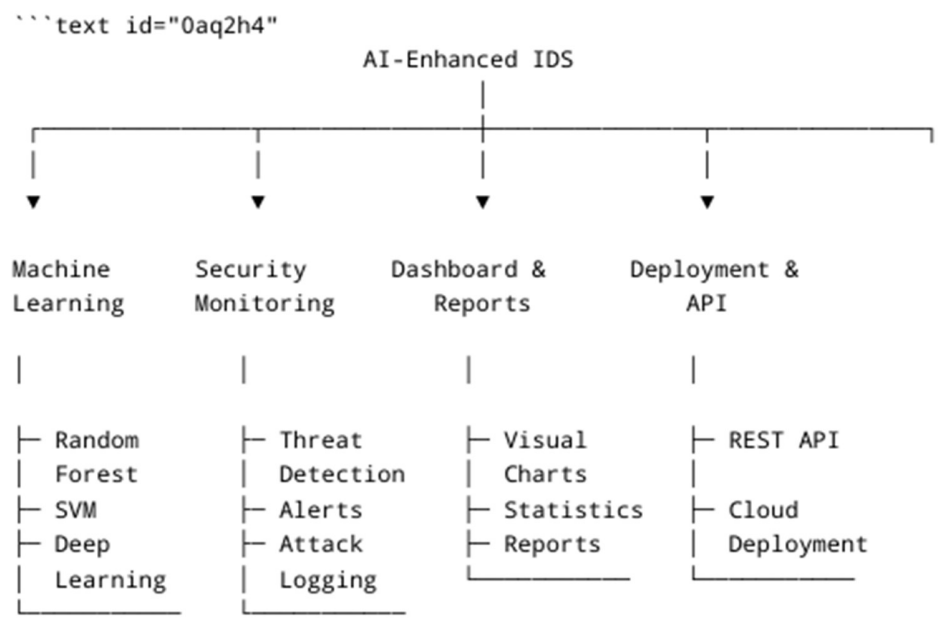


Figure 2: Brainstorming & Idea Grouping

2

Brainstorm

Write down any ideas that come to mind that address your problem statement. All ideas are valid — quantity over quality at this stage.

🕒 10 minutes

3

Group ideas

Take turns sharing your ideas while clustering similar or related notes as you go. In the last 10 minutes, give each cluster a sentence-like label.

🕒 20 minutes

Team Member 1 — ML & Model

Use Random Forest for multi-class attack detection

Train on NSL-KDD dataset (10 key features)

Save model as .pkl for fast Flask loading

Team Member 2 — Backend

Flask routes: /, /predict, /dashboard, /alerts, /api/predict

In-memory alert store with timestamp logging

JSON REST API endpoint for external integrations

Team Member 3 — Frontend

Bootstrap 5 responsive layout across all screens

Chart.js doughnut chart for live attack distribution

Color-coded threat badges (green=normal, red=attack)

Team Member 4 — Security

Demo shortcut buttons for all 5 attack classes

Confidence score display per prediction

Alert log as audit trail for forensic analysis

Team Member 5 — Enhancements

Live packet capture with Scapy/PyShark in future scope

Persistent DB (SQLite) to replace in-memory store

Email alerts via SMTP on high-confidence attacks

Team Member 6 — DevOps

Dockerise app with docker-compose for easy deploy

GitHub Actions CI pipeline for auto testing

requirements.txt for reproducible environments

Team Member 7 — Testing

Performance test: < 80ms API response latency

Unit tests for feature scaling and model inference

Test all 5 demo attack scenarios via API calls

Team Member 8 — Research

Compare Random Forest vs LSTM accuracy on NSL-KDD

Extend feature set from 10 to all 41 NSL-KDD features

Explore explainability with SHAP feature importance

Grouped Idea Clusters:

ML Core	Web Platform	User Interface	Integration & API	Future Scope
• Random Forest (100 trees) • NSL-KDD 10 features • StandardScaler normalisation • 5-class classification • Confidence scores	• Flask 3.1 routing • 6 HTTP endpoints • In-memory alert store • Pickle model loading • Startup model init	• Bootstrap 5 layout • Chart.js dashboard • Prediction form • Alert log table • Demo buttons (5 types)	• POST /api/predict • JSON I/O schema • Confidence flag • is_attack boolean • Timestamp in response	• Scapy live capture • SQLite persistence • SMTP email alerts • Docker deployment • SIEM integration

Step-3: Idea Prioritization

The generated ideas were evaluated based on:

- 1. Importance
- 2. Feasibility
- 3. Development Complexity
- 4. Resource Requirements
- 5. Expected Impact

Prioritization Table

Idea	Importance	Feasibility	Priority Rank
Random Forest Intrusion Detection	High	High	1
Real-Time Threat Monitoring	High	High	2
Automated Alert System	High	High	3
Dashboard Analytics	High	High	4
REST API Integration	Medium	High	5
Cloud Deployment	Medium	Medium	6
Threat Intelligence Integration	Medium	Medium	7
Deep Learning-Based Detection	High	Low	8

Diagram 3: Idea Prioritization Matrix

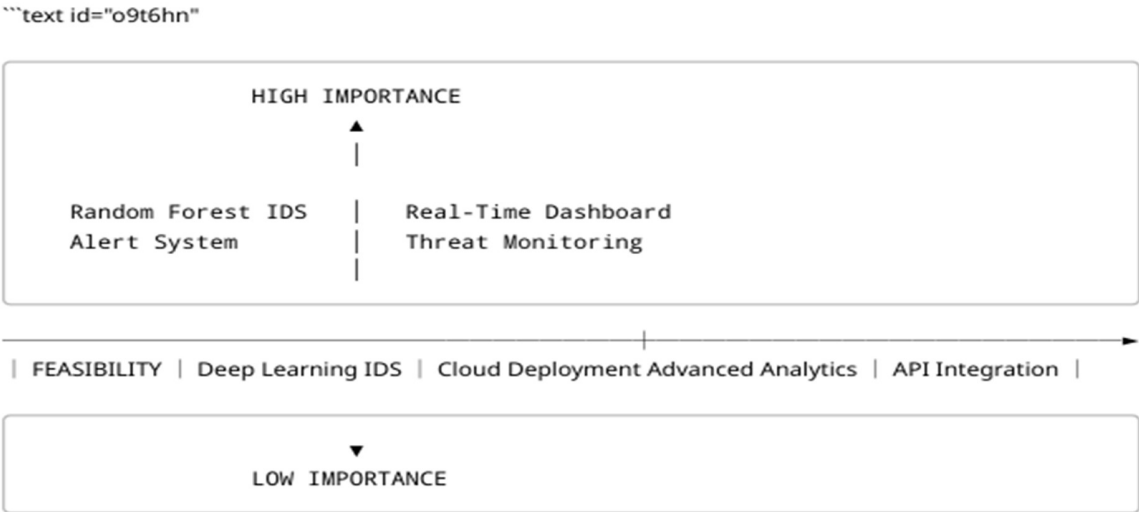


Figure 3: Idea Prioritization Matrix

← Less Feasible More Feasible →

High Importance	★ Prioritize — High Importance, High Feasibility ● ML model auto-classification of 5 attack types ● Flask web app with REST /api/predict endpoint ● Real-time confidence score per prediction ● Random Forest trained on NSL-KDD dataset	■ Plan — High Importance, Lower Feasibility ● Live packet capture integration (Scapy/PyShark) ● Automated model retraining pipeline ● Email / SMS alerting on high-confidence attacks ● Persistent database (SQLite / PostgreSQL)
Lower Importance	✓ Quick Wins — Lower Importance, High Feasibility ● Demo shortcut buttons for each attack category ● Bootstrap 5 responsive UI across devices ● In-memory alert log with timestamp display ● About page with API usage documentation	○ Backlog — Lower Importance, Lower Feasibility ● Deep learning (LSTM/CNN) model comparison ● SIEM platform integration (Splunk / QRadar) ● Dark/light theme toggle for dashboard ● Dockerised deployment with docker-compose

Priority Summary:

Priority Tier	Idea / Feature	Rationale
★ Must Have (High Importance, High Feasibility)	Random Forest ML model	Core deliverable — classifies 5 attack types with ~96-100% accuracy on NSL-KDD. Directly addresses the problem statement.
★ Must Have (High Importance, High Feasibility)	Flask web app + /api/predict	Provides both human-accessible UI and programmatic REST API. Lightweight and Python-native — very feasible to implement.
★ Must Have (High Importance, High Feasibility)	Real-time confidence scores	Adds interpretability beyond binary detection; easy to extract from model.predict_proba(). High value, low extra cost.
■ Should Have (High Importance, Lower Feasibility)	Live packet capture (Scapy)	Would enable real-world deployment but requires system-level access and significant added complexity. Deferred to future scope.
■ Should Have (High Importance, Lower Feasibility)	Persistent alert database	SQLite would make the tool production-ready. In-memory store is sufficient for the prototype phase.

Priority Tier	Idea / Feature	Rationale
Feasibility)		
✓ Quick Win (Lower Importance, High Feasibility)	Demo buttons for all 5 classes	Easy to implement; greatly improves demo experience without real network data. Good for showcase sessions.
✓ Quick Win (Lower Importance, High Feasibility)	Bootstrap 5 responsive UI	No additional code complexity; Bootstrap handles responsiveness out of the box.
○ Backlog (Lower Importance, Lower Feasibility)	LSTM/CNN model comparison	Interesting research direction but adds significant training complexity and is not required for the MVP deliverable.
○ Backlog (Lower Importance, Lower Feasibility)	SIEM platform integration	Enterprise-level enhancement best suited for post-project production hardening, not the current scope.

Selected Ideas for Implementation (Sprint Plan):

#	Feature	Description	Sprint	Status
1	ML Model Training	Random Forest on NSL-KDD; 5 classes; StandardScaler pipeline	Sprint 1–2	Completed
2	Flask Web Application	6 routes, Jinja2 templates, startup model loading	Sprint 3	Completed
3	Prediction Engine	10-feature form + /api/predict JSON endpoint	Sprint 3	Completed
4	Live Dashboard	Chart.js doughnut chart with real-time stats	Sprint 4	Completed
5	Alert Log System	In-memory store with timestamp, label, confidence	Sprint 4	Completed

#	Feature	Description	Sprint	Status
6	Demo Buttons	5 pre-filled attack scenarios for easy demonstration	Sprint 4	Completed
7	Confidence Scoring	Per-class probability from predict_proba(); displayed as %	Sprint 5	Completed
8	REST API Docs	About page with JSON schema and usage examples	Sprint 5	Completed

Final Selected Idea

Selected Solution

AI-Enhanced Intrusion Detection System

The selected solution utilizes a Random Forest Machine Learning model trained on the NSL-KDD dataset to classify network traffic into:

- Normal Traffic
- DoS Attacks
- Probe Attacks
- R2L Attacks
- U2R Attacks

The system includes:

- Real-Time Intrusion Detection
- Automated Alert Logging
- Dashboard Monitoring
- REST API Support
- Security Analytics

Diagram 4: Final Solution Selection

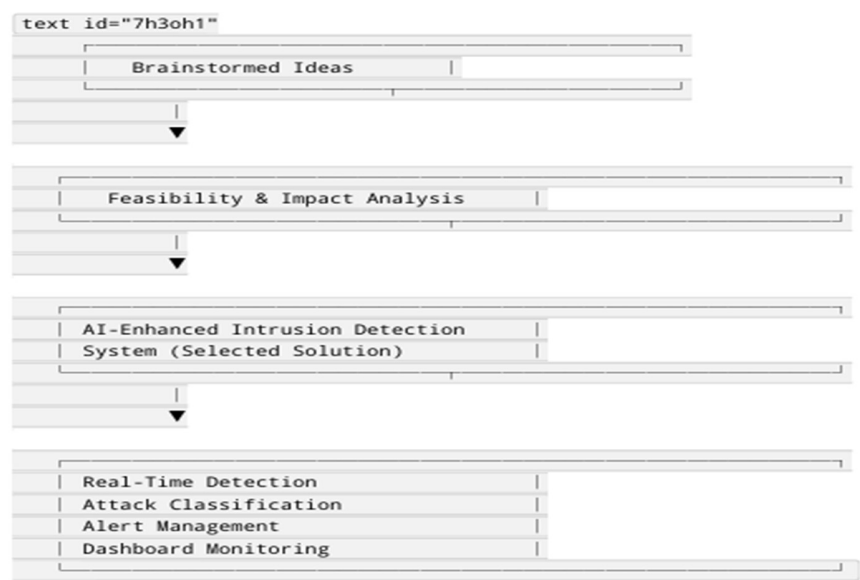


Figure 4: Final Selected Solution

Benefits of the Selected Solution

- High Detection Accuracy
 - Real-Time Threat Identification
 - Reduced False Positives
 - Automated Security Monitoring
 - Easy Integration with Existing Systems
 - Scalable Cloud Deployment Support
-

Conclusion

Through brainstorming, idea grouping, and prioritization, the team identified the AI-Enhanced Intrusion Detection System as the most feasible and impactful solution. The selected approach combines Artificial Intelligence and Cybersecurity to provide real-time network threat detection, automated alert generation, and enhanced protection against modern cyber attacks.